

AGENTIC AI

PROJECT REPORT

Multi-Agent Coding Assistant

Class ID	120055
Course	Agentic AI
Instructor	Dr. Farhan Mohsin
Submission Date	May 2025

Group Members

Student ID	Full Name	Role
14958	Shayan Akram	Team Lead
13905	Muhammad Ayan Husain	Developer
14908	Syed Muhammad Faseh	Developer

1. Introduction

The Multi-Agent Coding Assistant is an AI-powered system designed to fully automate the software development lifecycle. By orchestrating multiple specialized intelligent agents, the system autonomously generates code, executes and tests it, installs missing dependencies, researches current documentation, reviews code for security vulnerabilities, and produces technical documentation — all from a plain-language user prompt.

This project was developed as part of the Agentic AI course and simulates a real-world software development team where distinct AI agents collaborate in a pipeline, each handling a dedicated responsibility to produce high-quality, working code.

2. Objectives

The primary objectives of this project are:

- Develop a system that generates executable code from plain-language user prompts
- Implement automated multi-language code testing and execution
- Create an autonomous bug-fixing mechanism using AI-powered error analysis
- Integrate web research capabilities for up-to-date documentation retrieval
- Incorporate a security review agent to enforce best practices
- Auto-generate project documentation (README.md) for every task
- Demonstrate effective multi-agent collaboration in a real software development pipeline

3. System Architecture

The system is built on a modular multi-agent architecture where each agent is an independent async Python class. Agents communicate sequentially, passing structured data (JSON) between pipeline stages. The architecture supports both a command-line interface (main.py via Rich) and a web-based interface (app.py via Streamlit).

3.1 Agent Descriptions

The system consists of the following eight specialized agents:

Agent	Symbol	Responsibility
Planner Agent	□	Analyzes user prompts and produces a structured, step-by-step architectural plan
Code Agent	□	Generates complete, multi-file executable code based on the architectural plan
Test Agent	□	Executes code in a sandboxed workspace; supports Python, JavaScript, and C++
Fix Agent	□	Diagnoses execution errors and regenerates corrected code automatically
Dependency Agent	□	Detects ModuleNotFoundError and auto-installs missing pip packages
Research Agent	□	Queries DuckDuckGo for up-to-date documentation before code generation

Review Agent	☐	Audits generated code for security vulnerabilities and best practice violations
Docs Agent	☐	Generates a comprehensive README.md for each completed project

3.2 Execution Pipeline

The system follows a structured, sequential pipeline:

- User Input — Natural language task prompt entered via CLI or web interface
- Research Phase (optional) — ResearchAgent fetches relevant online documentation
- Planning Phase — PlannerAgent generates a detailed architectural blueprint
- Code Generation — CodeAgent produces structured JSON output with all required files
- Security Review (optional) — ReviewAgent audits and refactors the generated code
- Test & Fix Loop — TestAgent executes code; on failure, DependencyAgent or FixAgent intervenes (up to 3 retries)
- Documentation — DocsAgent produces a README.md for successful projects
- History & Export — All artifacts are saved with timestamps and downloadable as ZIP

4. Tools & Technologies

4.1 Core Technologies

Technology	Purpose / Notes
Python 3.10+	Core programming language; asyncio for non-blocking agent execution
Ollama (Local LLMs)	Runs models like Mistral, CodeLlama, LLaMA3 fully locally — no API key required
Streamlit	Web-based interactive UI with real-time status updates and project history
Rich	Terminal-based CLI with syntax highlighting and formatted output panels
aiohttp	Asynchronous HTTP client for communicating with the Ollama API
duckduckgo-search	Privacy-respecting web search library for the Research Agent
python-dotenv	Environment variable management for model and API URL configuration
asyncio / subprocess	Asynchronous code execution and process management for the Test Agent

5. Key Features

- Multi-language support — generates and tests Python, JavaScript, and C++ code
- Fully local execution — Ollama LLMs run on-device; no internet or API key required for AI inference
- Agentic autonomy — agents independently decide, retry, and recover without human intervention
- Persistent project history — all sessions saved with timestamps and downloadable as ZIP archives
- Optional web research — pulls live documentation from DuckDuckGo before code generation
- Security-aware — dedicated ReviewAgent checks for vulnerabilities before execution

- Dual interface — identical functionality available via terminal (Rich) and browser (Streamlit)

6. Limitations

- Performance and output quality depend on the local hardware (RAM/GPU) available for Ollama
- Complex, multi-module projects may exceed the LLM context window and produce incomplete outputs
- The Test Agent is limited to a 15-second execution timeout per attempt to prevent infinite loops
- Web research depends on DuckDuckGo availability and may occasionally return irrelevant results
- C++ compilation requires g++ to be installed separately on the host system
- Code generation is non-deterministic; results may vary across identical prompts

7. Future Enhancements

- Integration with GitHub for automated repository creation and version-controlled commits
- Support for additional programming languages (Java, Go, Rust, TypeScript)
- A dedicated UI/UX Agent capable of generating complete frontend web applications
- Cloud deployment of the Streamlit interface for team-wide collaborative access
- Persistent conversation memory across sessions for iterative multi-turn project development
- Integration of code linters (Pylint, ESLint) into the review pipeline for static analysis

8. Conclusion

The Multi-Agent Coding Assistant successfully demonstrates how multiple specialized AI agents can collaborate autonomously to automate the full software development lifecycle. From parsing a plain-language prompt to producing tested, documented, and security-reviewed code, the system showcases the practical power of agentic AI architectures.

This project provided hands-on experience in designing asynchronous multi-agent pipelines, prompt engineering for structured JSON outputs, and building production-quality AI-driven tools. The modular design ensures each agent can be independently upgraded, making the system a strong foundation for more advanced agentic applications.